

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №4
Дисциплина: Теория параллельных алгоритмов

Работу выполнил: _____ Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин Е.А.

Краснодар
2026

Тема. Сортировка методом пузырька, сортировка Шелла, быстрая сортировка.

Цель. Распараллелить процесс сортировки методом пузырька, сортировки Шелла, быстрой сортировки.

Задание. Составить последовательный алгоритм сортировки методом пузырька, сортировки Шелла, быстрой сортировки. Построить графы алгоритмов. Исследовать на них зависимости и возможности распараллеливания. Выполнить параллельную реализацию алгоритмов. Составить таблицу, отражающую сравнительное время выполнения последовательных и параллельных алгоритмов на 2-х, 3-х и 4-х ядрах.

1. Сортировка пузырьком (Bubble Sort) и её параллельный вариант.

Последовательный алгоритм многократно проходит по массиву, сравнивая соседние элементы и меняя их местами, если они расположены в неправильном порядке. Максимальное количество проходов – $n-1$. Сложность $O(n^2)$.

Параллельный вариант использует четно-нечетную сортировку. На чётных фазах сравниваются пары $(0,1),(2,3),\dots$, на нечётных – $(1,2),(3,4),\dots$. Все сравнения в пределах одной фазы независимы, что позволяет распараллелить каждую фазу с помощью `#pragma omp parallel for`. Количество фаз – n .

2. Сортировка Шелла (Shell Sort).

Последовательный алгоритм – улучшенная версия сортировки вставками. Элементы сравниваются и перемещаются на расстоянии (шаг, *gap*), постепенно уменьшая шаг до 1. Сложность зависит от выбора последовательности шагов (в работе используется последовательность Шелла: $n/2, n/4, \dots, 1$). Обычно $O(n^{3/2})$ или лучше.

Параллельный вариант для каждого значения *gap* разбивает массив на *gap* независимых подпоследовательностей:

$(0, \text{gap}, 2\text{gap}, \dots), (1, 1+\text{gap}, 1+2\text{gap}, \dots), \dots$

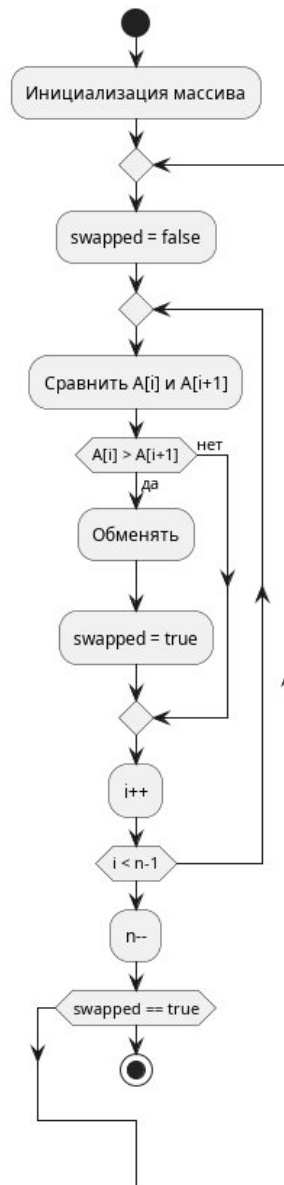
Каждая подпоследовательность сортируется методом вставок. Поскольку подпоследовательности независимы, их можно обрабатывать параллельно с помощью `#pragma omp parallel for`.

3. Быстрая сортировка (Quick Sort).

Последовательный алгоритм выбирает опорный элемент, разбивает массив на две части (элементы меньше опорного и больше либо равные) и рекурсивно сортирует каждую часть. Средняя сложность $O(n \log n)$, худшая $O(n^2)$.

Параллельный вариант использует задачи OpenMP (`#pragma omp task`). После разбиения массива рекурсивные вызовы для левой и правой частей оформляются как задачи, которые могут выполняться параллельно доступными потоками. Для ограничения глубины параллелизма используется порог, при котором сортировка переключается на последовательную реализацию.

Последовательный алгоритм сортировки методом пузырька можно схематично изобразить в следующем виде:



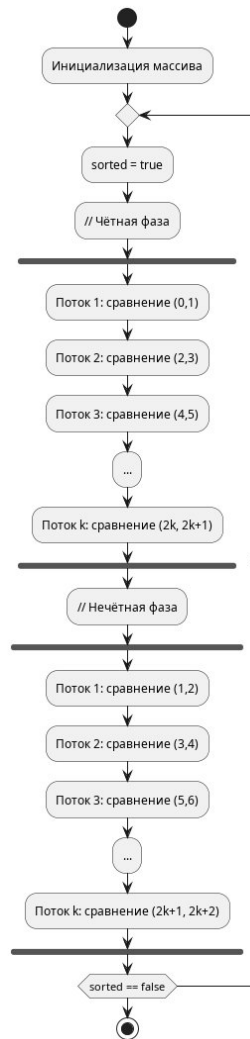
Реализуем последовательный алгоритм:

```

void bubble_sort_seq(vector<int>& arr) {
    int n = arr.size();
    bool swapped;
    for (int i = 0; i < n-1; ++i) {
        swapped = false;
        for (int j = 0; j < n-i-1; ++j) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

```

Параллельный алгоритм можно схематично изобразить в следующем виде:

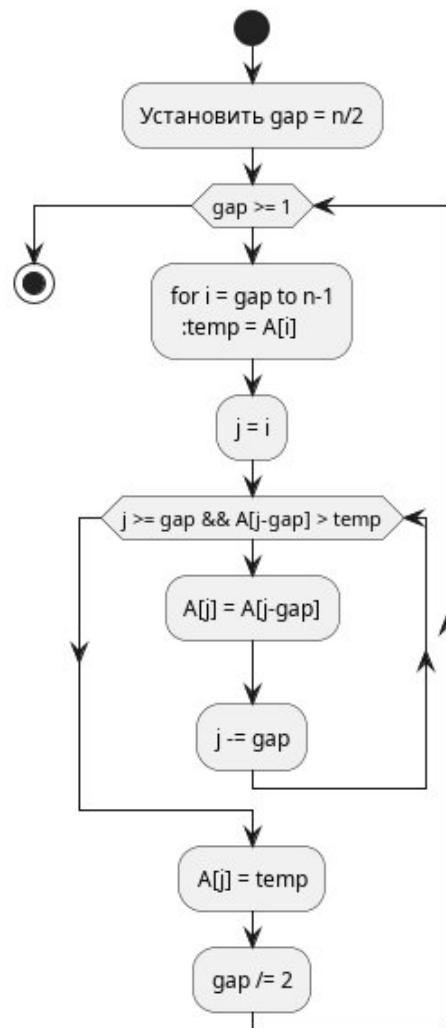


Реализуем параллельный алгоритм:

```

void bubble_sort_par(vector<int>& arr, int num_threads) {
    omp_set_num_threads(num_threads);
    int n = arr.size();
    bool sorted = false;
    while (!sorted) {
        sorted = true;
        #pragma omp parallel for
        for (int i = 0; i < n-1; i += 2) {
            if (arr[i] > arr[i+1]) {
                swap(arr[i], arr[i+1]);
                sorted = false;
            }
        }
        #pragma omp parallel for
        for (int i = 1; i < n-1; i += 2) {
            if (arr[i] > arr[i+1]) {
                swap(arr[i], arr[i+1]);
                sorted = false;
            }
        }
    }
}
  
```

Последовательный алгоритм сортировки Шелла можно схематично изобразить в следующем виде:

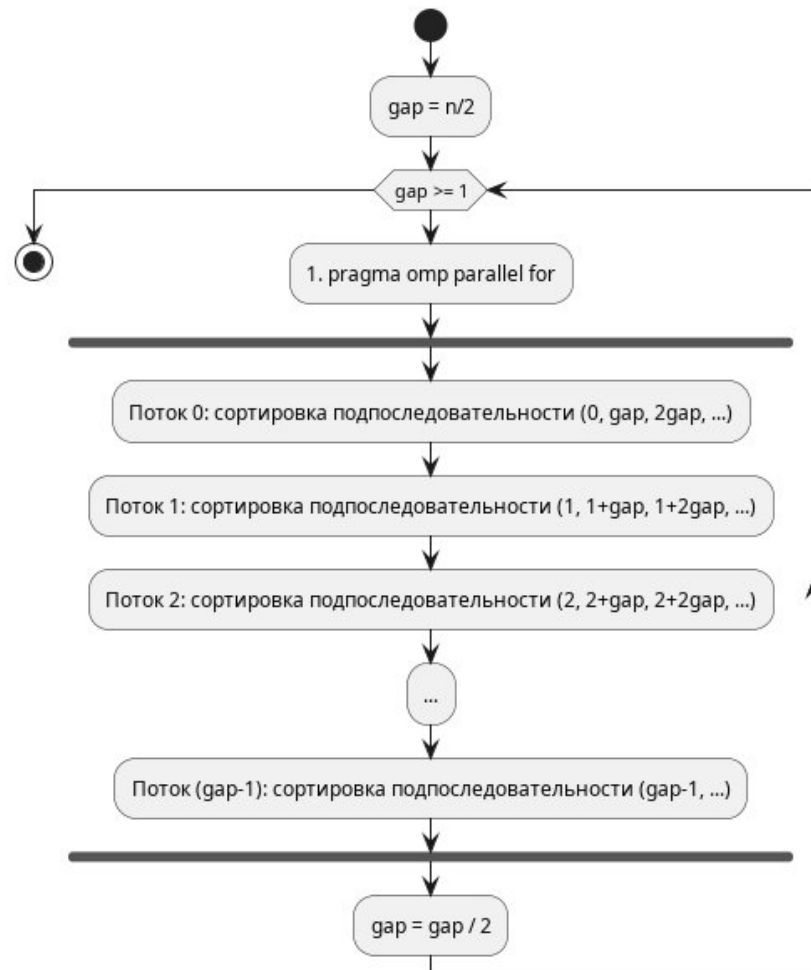


Реализуем последовательный алгоритм:

```

void shell_sort_seq(vector<int>& arr) {
    int n = arr.size();
    for (int gap = n/2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; ++i) {
            int temp = arr[i];
            int j = i;
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }
            arr[j] = temp;
        }
    }
}
  
```

Параллельный алгоритм можно схематично изобразить в следующем виде:



Реализуем параллельный алгоритм:

```

void shell_sort_par(vector<int>& arr, int num_threads) {
    omp_set_num_threads(num_threads);
    int n = arr.size();
    for (int gap = n/2; gap > 0; gap /= 2) {
        #pragma omp parallel for
        for (int k = 0; k < gap; ++k) {
            for (int i = k + gap; i < n; i += gap) {
                int temp = arr[i];
                int j = i;
                while (j >= gap && arr[j - gap] > temp) {
                    arr[j] = arr[j - gap];
                    j -= gap;
                }
                arr[j] = temp;
            }
        }
    }
}
  
```

Последовательный алгоритм быстрой сортировки можно схематично изобразить в следующем виде:

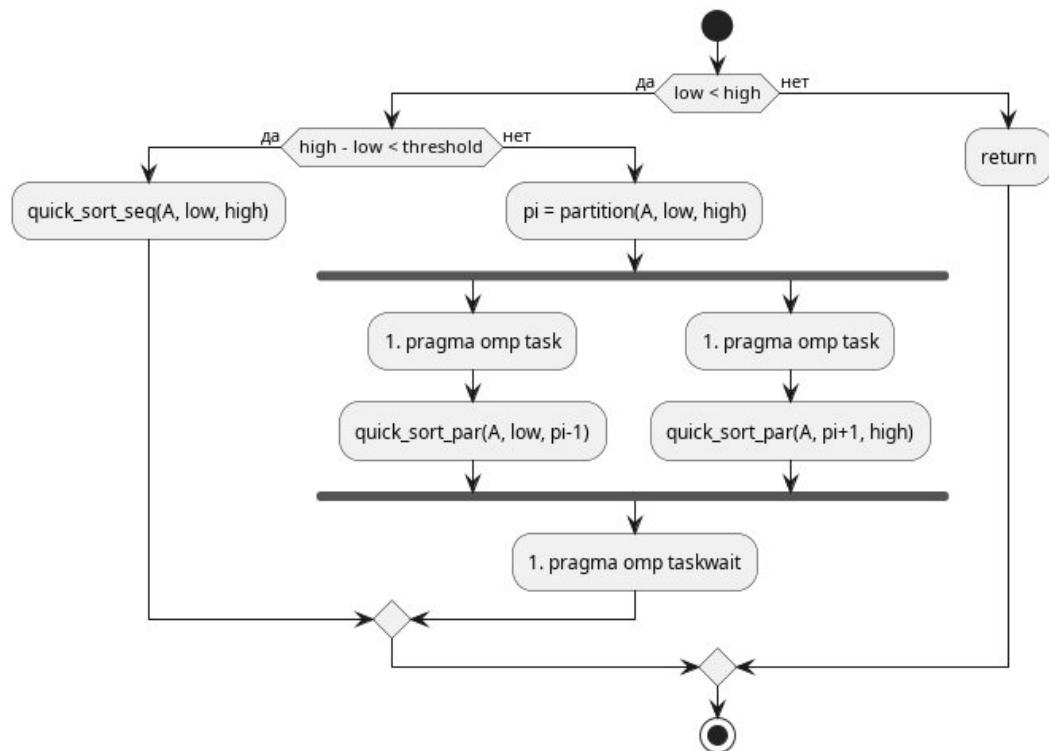


Реализуем последовательный алгоритм:

```
int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; ++j) {
        if (arr[j] ≤ pivot) {
            ++i;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[high]);
    return i+1;
}

void quick_sort_seq(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quick_sort_seq(arr, low, pi-1);
        quick_sort_seq(arr, pi+1, high);
    }
}
```

Параллельный алгоритм можно схематично изобразить в следующем виде:



Реализуем параллельный алгоритм:

```

void quick_sort_par(vector<int>& arr, int low, int high, int threshold) {
    if (low < high) {
        if (high - low < threshold) {
            quick_sort_seq(arr, low, high);
            return;
        }
        int pi = partition(arr, low, high);
        #pragma omp task
        quick_sort_par(arr, low, pi-1, threshold);
        #pragma omp task
        quick_sort_par(arr, pi+1, high, threshold);
        #pragma omp taskwait
    }
}

void quick_sort_par_wrapper(vector<int>& arr, int num_threads) {
    omp_set_num_threads(num_threads);
    #pragma omp parallel
    {
        #pragma omp single
        quick_sort_par(arr, 0, arr.size()-1, SEQUENTIAL_THRESHOLD);
    }
}

```

Полный код итоговой программы:

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <chrono>
#include <iomanip>
#include <algorithm>
#include <omp.h>

using namespace std;
using namespace chrono;

const int MAX_THREADS = 4;
const int SEQUENTIAL_THRESHOLD = 1000;

void bubble_sort_seq(vector<int>& arr) {
    int n = arr.size();
    bool swapped;
    for (int i = 0; i < n-1; ++i) {
        swapped = false;
        for (int j = 0; j < n-i-1; ++j) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

void bubble_sort_par(vector<int>& arr, int num_threads) {
    omp_set_num_threads(num_threads);
    int n = arr.size();
    bool sorted = false;
    while (!sorted) {
        sorted = true;
        #pragma omp parallel for
        for (int i = 0; i < n-1; i += 2) {
            if (arr[i] > arr[i+1]) {
                swap(arr[i], arr[i+1]);
                sorted = false;
            }
        }
        #pragma omp parallel for
        for (int i = 1; i < n-1; i += 2) {
            if (arr[i] > arr[i+1]) {
                swap(arr[i], arr[i+1]);
                sorted = false;
            }
        }
    }
}

void shell_sort_seq(vector<int>& arr) {
    int n = arr.size();
    for (int gap = n/2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; ++i) {
            int temp = arr[i];
            int j = i;
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }
            arr[j] = temp;
        }
    }
}

void shell_sort_par(vector<int>& arr, int num_threads) {
    omp_set_num_threads(num_threads);
    int n = arr.size();
    for (int gap = n/2; gap > 0; gap /= 2) {
        #pragma omp parallel for
        for (int k = 0; k < gap; ++k) {
            for (int i = k + gap; i < n; i += gap) {
                int temp = arr[i];
                int j = i;
```

```

        while (j ≥ gap && arr[j - gap] > temp) {
            arr[j] = arr[j - gap];
            j -= gap;
        }
        arr[j] = temp;
    }
}

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; ++j) {
        if (arr[j] ≤ pivot) {
            ++i;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[high]);
    return i+1;
}

void quick_sort_seq(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quick_sort_seq(arr, low, pi-1);
        quick_sort_seq(arr, pi+1, high);
    }
}

void quick_sort_par(vector<int>& arr, int low, int high, int threshold) {
    if (low < high) {
        if (high - low < threshold) {
            quick_sort_seq(arr, low, high);
            return;
        }
        int pi = partition(arr, low, high);
        #pragma omp task
        quick_sort_par(arr, low, pi-1, threshold);
        #pragma omp task
        quick_sort_par(arr, pi+1, high, threshold);
        #pragma omp taskwait
    }
}

void quick_sort_par_wrapper(vector<int>& arr, int num_threads) {
    omp_set_num_threads(num_threads);
    #pragma omp parallel
    {
        #pragma omp single
        quick_sort_par(arr, 0, arr.size()-1, SEQUENTIAL_THRESHOLD);
    }
}

double measure_time_seq(void (*sort_func)(vector<int>&), vector<int> arr) {
    auto start = high_resolution_clock::now();
    sort_func(arr);
    auto end = high_resolution_clock::now();
    return duration_cast<milliseconds>(end - start).count();
}

double measure_time_par(void (*sort_func)(vector<int>&, int), vector<int> arr, int num_threads) {
    auto start = high_resolution_clock::now();
    sort_func(arr, num_threads);
    auto end = high_resolution_clock::now();
    return duration_cast<milliseconds>(end - start).count();
}

double measure_quick_seq(vector<int> arr) {
    auto start = high_resolution_clock::now();
    quick_sort_seq(arr, 0, arr.size()-1);
    auto end = high_resolution_clock::now();
    return duration_cast<milliseconds>(end - start).count();
}

double measure_quick_par(vector<int> arr, int num_threads) {
    auto start = high_resolution_clock::now();
    quick_sort_par_wrapper(arr, num_threads);
}

```

```

    auto end = high_resolution_clock::now();
    return duration_cast<milliseconds>(end - start).count();
}

void run_benchmark() {
    vector<int> sizes = {10'000, 50'000, 100'000, 500'000};
    vector<int> threads = {2, 3, 4};

    cout << "\n===== BUBBLE SORT =====\n";
    cout << setw(10) << "Size" << setw(15) << "Threads" << setw(20) << "Seq (ms)" << setw(20) << "Par (ms)" <<
    setw(15) << "Speedup" << endl;
    for (int n : sizes) {
        vector<int> base(n);
        for (int i = 0; i < n; ++i) base[i] = rand() % 100000;
        double time_seq = measure_time_seq(bubble_sort_seq, base);
        for (int t : threads) {
            double time_par = measure_time_par(bubble_sort_par, base, t);
            double speedup = time_seq / time_par;
            cout << setw(10) << n << setw(15) << t << setw(20) << time_seq << setw(20) << time_par << setw(15)
            << fixed << setprecision(2) << speedup << endl;
        }
        cout << "-----\n";
    }

    cout << "\n===== SHELL SORT =====\n";
    cout << setw(10) << "Size" << setw(15) << "Threads" << setw(20) << "Seq (ms)" << setw(20) << "Par (ms)" <<
    setw(15) << "Speedup" << endl;
    for (int n : sizes) {
        vector<int> base(n);
        for (int i = 0; i < n; ++i) base[i] = rand() % 100000;
        double time_seq = measure_time_seq(shell_sort_seq, base);
        for (int t : threads) {
            double time_par = measure_time_par(shell_sort_par, base, t);
            double speedup = time_seq / time_par;
            cout << setw(10) << n << setw(15) << t << setw(20) << time_seq << setw(20) << time_par << setw(15)
            << fixed << setprecision(2) << speedup << endl;
        }
        cout << "-----\n";
    }

    cout << "\n===== QUICK SORT =====\n";
    cout << setw(10) << "Size" << setw(15) << "Threads" << setw(20) << "Seq (ms)" << setw(20) << "Par (ms)" <<
    setw(15) << "Speedup" << endl;
    for (int n : sizes) {
        vector<int> base(n);
        for (int i = 0; i < n; ++i) base[i] = rand() % 100000;
        double time_seq = measure_quick_seq(base);
        for (int t : threads) {
            double time_par = measure_quick_par(base, t);
            double speedup = time_seq / time_par;
            cout << setw(10) << n << setw(15) << t << setw(20) << time_seq << setw(20) << time_par << setw(15)
            << fixed << setprecision(2) << speedup << endl;
        }
        cout << "-----\n";
    }
}

int main() {
    srand(time(nullptr));
    run_benchmark();
    return 0;
}

```

Зафиксируем показатели работы алгоритмов:

===== BUBBLE SORT =====				
Size	Threads	Seq (ms)	Par (ms)	Speedup
10000	2	281	97	2.90
10000	3	281.00	131.00	2.15
10000	4	281.00	212.00	1.33

50000	2	9411.00	1172.00	8.03
50000	3	9411.00	1147.00	8.20
50000	4	9411.00	1175.00	8.01

100000	2	38737.00	5686.00	6.81
100000	3	38737.00	5592.00	6.93
100000	4	38737.00	4166.00	9.30

500000	2	1011903.00	166774.00	6.07
500000	3	1011903.00	240475.00	4.21
500000	4	1011903.00	232468.00	4.35

===== SHELL SORT =====				
Size	Threads	Seq (ms)	Par (ms)	Speedup
10000	2	2.00	1.00	2.00
10000	3	2.00	1.00	2.00
10000	4	2.00	1.00	2.00

50000	2	14.00	13.00	1.08
50000	3	14.00	10.00	1.40
50000	4	14.00	11.00	1.27

100000	2	39.00	25.00	1.56
100000	3	39.00	29.00	1.34
100000	4	39.00	24.00	1.62

500000	2	187.00	67.00	2.79
500000	3	187.00	80.00	2.34
500000	4	187.00	82.00	2.28

===== QUICK SORT =====				
Size	Threads	Seq (ms)	Par (ms)	Speedup
10000	2	1.00	1.00	1.00
10000	3	1.00	1.00	1.00
10000	4	1.00	1.00	1.00

50000	2	8.00	12.00	0.67
50000	3	8.00	10.00	0.80
50000	4	8.00	8.00	1.00

100000	2	17.00	77.00	0.22
100000	3	17.00	73.00	0.23
100000	4	17.00	47.00	0.36

500000	2	95.00	1251.00	0.08
500000	3	95.00	781.00	0.12
500000	4	95.00	1564.00	0.06

Вывод. Мы рассмотрели сортировку методом пузырька, сортировку Шелла, быструю сортировку, изобразили их схематично, после чего, изучив возможности распараллеливания, выполнили замеры времени для последовательных и параллельных случаев на 2-х, 3-х и 4-х потоках.